

OOP REPORT

ABSTRACT

This report outlines the formal software architecture and design parameters for the Smart Library Management System (SmartLib). SmartLib is a library management system developed to demonstrate how object-oriented programming concepts can be applied in a practical software project.

The program was executed using Java 17+ and Maven and was developed in phases with each phase targeting a different area of software engineering. These areas included domain modelling, SOLID principles, design patterns, generics, functional programming, concurrency, and modern Java features. During the implementation, emphasis was placed on utilizing best practices in oop software development. maintain, and extend. Features such as policy-based borrowing, event-driven communication, immutable records, and asynchronous processing were incorporated to improve flexibility and reliability. Testing, UML diagrams, and JaCoCo reports were also used to evaluate the quality of the final solution.

Introduction and Design Overview

The project was designed to simulate the operations of a small library while demonstrating latest software development practices. The core function of the design was centered around managing books, members, loans, reservations, fines, notifications, analytics, and inventory. Although the project is relatively small, it addresses many challenges commonly found in real-world software systems, such as object creation, modular design, event handling, and concurrency management. The primary domain components track the programmatic life cycle of Books, Members, Loans, Reservations, Fines, and Notifications under strict, multi-tiered business constraints. To make development more manageable, the project was divided into six phases. A step-by-step methodology was adopted to make the code and application easier to run and test. We integrated components into real modules for the system. At the pivot was the architecture domain layer, which represents the core business entities.

SOLID Analysis and Refactoring

The LibraryManager class implementation exemplifies a classic architectural design that attempts to trigger multiple unrelated components within a single structural block. This violates all five core SOLID principles. One of the most important improvements made during the development of SmartLib was the adoption of the SOLID design principles. Initially, responsibilities were concentrated in a few classes, which made the design harder

to maintain. Through refactoring, responsibilities were distributed among specific classes, services and abstractions. The Single Responsibility Principle was applied by assigning separate tasks to different components, such as loan management, fine calculation, notifications, and data persistence. The Open/Closed Principle was implemented through the borrowing rules, allowing new membership types to be added without modifying existing code. The Liskov Substitution Principle was achieved through the use of interfaces and abstractions, ensuring that alternative implementations could be used interchangeably. The Interface Segregation Principle encouraged the creation of focused interfaces with only the methods required by clients. Finally, the Dependency Inversion Principle was applied through constructor injections and abstraction-based design, improving flexibility and testability.

The Builder pattern was used through the LoanBuilder class, making object creation clearer and reducing the risk of incomplete object construction. The Decorator pattern was applied in the notification system, where additional features such as logging, retries, and rate limiting could be added without modifying the core notification service. The Observer pattern was implemented through the LibraryEventBus, allowing different parts of the system to react to events independently. For example, returning a book can trigger notifications, fine calculations, and audit logging without creating tight dependencies between components.

The SmartLib project also explored Java generics and their role in building reusable and type-safe software. The Catalogue class uses bounded generic parameters to ensure that only suitable objects can be stored and managed. Wildcards based on the PECS principle were introduced to improve flexibility while maintaining type safety. Another area of focus was type erasure, which removes generic type information during compilation. Reflection examples and a TokenType implementation were used to demonstrate both the limitations of type erasure and techniques for working around them when runtime type information is required. Overall, this phase highlighted how generics help developers express constraints and improve reliability.

The Stream API was introduced to simplify data processing and make the code more expressive. Instead of relying heavily on loops, SmartLib uses streams to perform filtering, grouping, mapping, and aggregation operations. This approach improved readability and reduced the amount of boilerplate code. A custom LoanSummaryCollector was also developed to demonstrate how Java combines functional and practical programming approaches. Although mutable accumulation was used internally for efficiency, the final output remained immutable. Additional functional programming features such as method

references and predicate composition were used to encourage code reuse and improve clarity.

Concurrency was introduced to improve responsiveness while ensuring the system remained reliable. The `BookReturnProcessor` uses `CompletableFuture` and a fixed thread pool to process book returns asynchronously. The workflow is divided into clear stages, including validation, state updates, notifications, and event publishing. Special attention was given to protecting shared data. Critical updates involving inventory, loan records, and fines are performed within protected sections to prevent inconsistencies. Atomic Integer counters were used to track successful and failed operations efficiently, while `ReentrantReadWriteLock` and `Condition` were used for inventory management. This phase reinforced the importance of careful state management when developing concurrent applications.

Java extensions were integrated throughout the project to improve readability, maintainability, and reliability. Records were used to represent immutable data structures; sealed classes defined a real controlled hierarchy of loan results. Pattern matching reduced bottle necks code and made decision-making logic easier. several expressions provided a cleaner alternative to traditional conditional statements, particularly when selecting borrowing policies. Code blocks were simplified during the creation of JSON payloads, and the Java Platform Module System (JPMS) strengthened modularity by making dependencies and architectural boundaries more explicit.

The testing class played a major role in the development process. All cell block codes were tested using the test class. JaCoCo reports demonstrate strong lineage across the application, with most business logic, architectural components, and core features tested through automated tests. The results shows the system is stable, reliable, and capable of handling the majority of expected scenarios.

In summary, the use of SOLID design principles shifts the system from an unsustainable monolith to an agile, modular architecture. By breaking up the original `LibraryManager` class into clearly distinguishing data structures and abstract behavior pipelines, the code becomes trivial to extend, isolate, and continuously test to enterprise standards.

Implementing the project provided valuable experience in understanding object-oriented design principles and modern Java technologies. The phased development approach made it easier to grasp the concept of software development.

Question 3.2: Explain what type of erasure is and why the following code produces a compile-time warning rather than an error:

```
Catalogue <Book> bookCat = new Catalogue <>();
```

```
Catalogue raw = bookCat ; // raw type  
raw.add("not a book "); // heap pollution
```

Response:

Type erasure is the process used in Java Generics where generic type information is removed during compilation. From the above code for instance, the line `Catalogue <Book> bookCat = new Catalogue <>();`, will be treated at run time as `Catalogue bookCat= new Catalogue();`.

References:

1. https://mrcet.com/downloads/digital_notes/CSE/II%20Year/CS/OBJECT%20ORIENTED%20PROGRAMMING%20THROUGH%20JAVA.pdf
2. https://www.ms.sapientia.ro/~manyi/teaching/oop/oop_java.pdf
3. https://lecture-notes.tiu.edu.iq/wp-content/uploads/2021/10/An-Introduction-to-Object-Oriented-Programming-with-Java-by-C.-Thomas-Wu-z-lib.org_.pdf
4. <https://www.jbiet.edu.in/pdf/fls/ECM-Coursematerial/OOP-NOTES-final.pdf>
5. <https://library.uc.edu.kh/userfiles/pdf/8.Fundamentals%20of%20OOP%20and%20Data%20Structures%20in%20Java.pdf>

Declaration

By submitting this assignment, I certify that:

- This work is solely our own.
- All SQL scripts are original or properly attributed.
- I have not shared my solution with any other student.
- I understand the consequences of academic dishonesty as outlined in the University's student conduct policy.

Students:

Jusufu Henry Swaray

Kadiatu Kanu